COMPUTER ENGINEERING

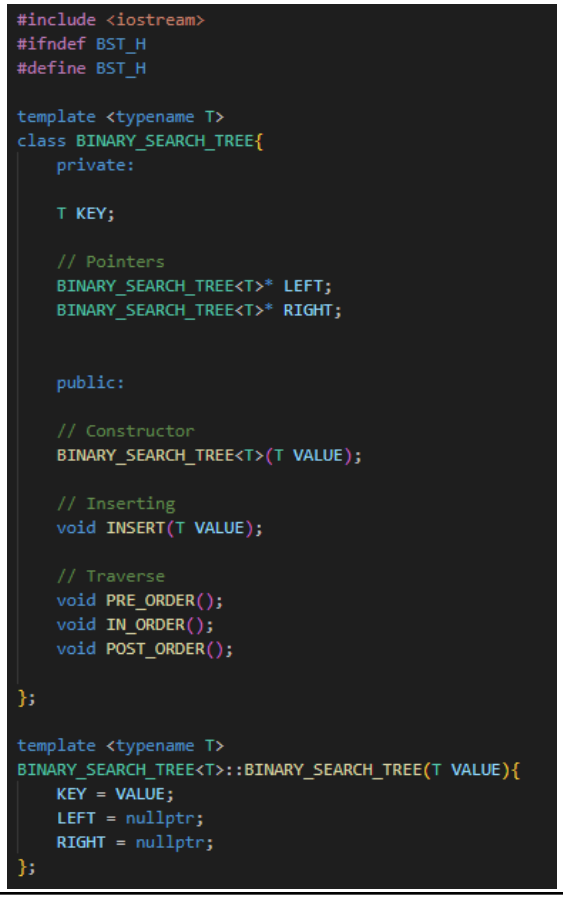
QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	18/09/2026	**DATE SUBMITTED:**	
NAME:	Quiming, Kadaphy James A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. HOA 9.1 TREE ADT

1. PROCEDURE OUTPUT

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODE	ANALYSIS
	Header Code Block

```cpp
#include <iostream>
#ifndef BST_H
#define BST_H

template <typename T>
class BINARY_SEARCH_TREE{
    private:

    T KEY;

    // Pointers
    BINARY_SEARCH_TREE<T>* LEFT;
    BINARY_SEARCH_TREE<T>* RIGHT;


    public:

    // Constructor
    BINARY_SEARCH_TREE<T>(T VALUE);

    // Inserting
    void INSERT(T VALUE);

    // Traverse
    void PRE_ORDER();
    void IN_ORDER();
    void POST_ORDER();

};

template <typename T>
BINARY_SEARCH_TREE<T>::BINARY_SEARCH_TREE(T VALUE){
    KEY = VALUE;
    LEFT = nullptr;
    RIGHT = nullptr;
};
```

```cpp
template <typename T>
void BINARY_SEARCH_TREE<T>::INSERT(T VALUE){
    if(VALUE < KEY){
        if(LEFT == nullptr){
            LEFT = new BINARY_SEARCH_TREE<T>(VALUE);
        }else{
            LEFT->INSERT(VALUE);
        }
    }
    else if(VALUE > KEY){
        if(RIGHT == nullptr){
            RIGHT = new BINARY_SEARCH_TREE<T>(VALUE);
        }else{
            RIGHT->INSERT(VALUE);
        }
    }
};
```

Inserts a child if that spot is not nullptr, skips another

```cpp
template <typename T>
void BINARY_SEARCH_TREE<T>::PRE_ORDER(){
    std::cout << KEY << "    ";
    if(LEFT != nullptr){
        LEFT -> PRE_ORDER();
    }

    if(RIGHT != nullptr){
        RIGHT -> PRE_ORDER();
    }
}
```

```cpp
template <typename T>
void BINARY_SEARCH_TREE<T>::IN_ORDER(){
    if(LEFT != nullptr){
        LEFT -> IN_ORDER();
    }

    std::cout << KEY << "    ";

    if(RIGHT != nullptr){
        RIGHT -> IN_ORDER();
    }
}
```

```
template <typename T>
void BINARY_SEARCH_TREE<T>::POST_ORDER(){
    if(LEFT != nullptr){
        LEFT -> POST_ORDER();
    }


    if(RIGHT != nullptr){
        RIGHT -> POST_ORDER();
    }

    std::cout << KEY << "    ";
}
```

```
// Create the root node
BINARY_SEARCH_TREE<int> TREE(20);

TREE.INSERT(10);
TREE.INSERT(5);
TREE.INSERT(70);
TREE.INSERT(80);
TREE.INSERT(40);
TREE.INSERT(50);

std::cout << "PRE ORDER: ";
TREE.PRE_ORDER();
std::cout << std::endl;

std::cout << "IN ORDER: ";
TREE.IN_ORDER();
std::cout << std::endl;

std::cout << "POST ORDER: ";
TREE.POST_ORDER();
std::cout << std::endl;
```

FINAL OUTPUT

```
PRE ORDER: 20   10   5   70   40   50   80
IN ORDER: 5   10   20   40   50   70   80
POST ORDER: 5   10   50   40   80   70   20
```

CONCLUSION

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic

work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.